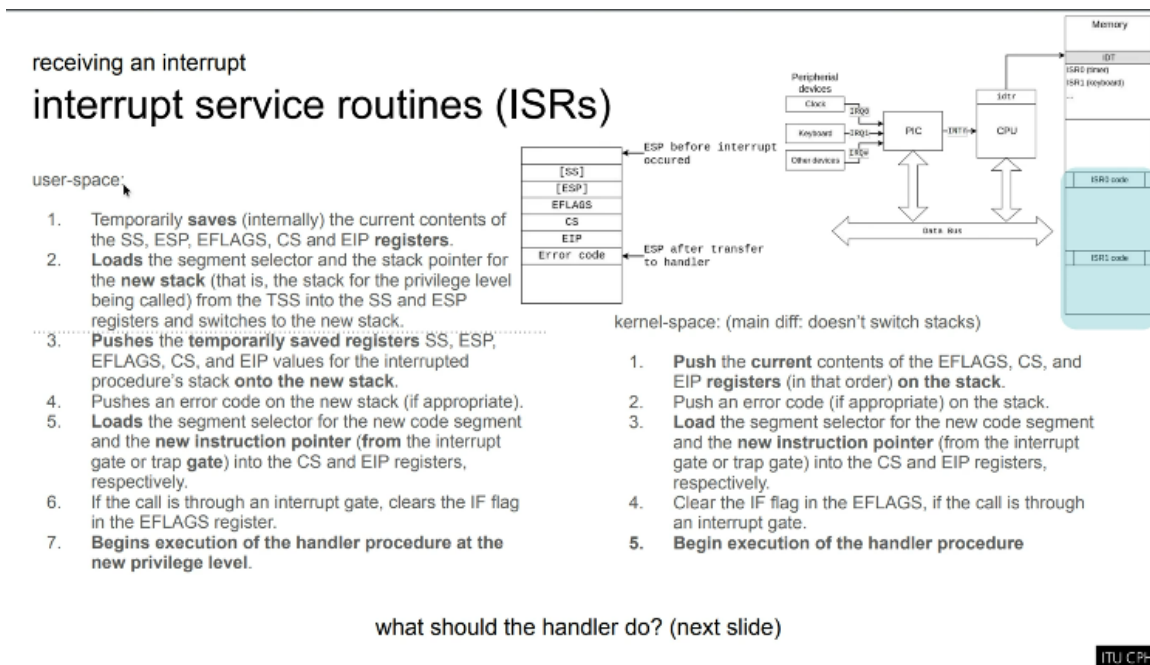


- The `idtr` (interrupt descriptor table register) is a register in the CPU which points at some region of memory where the interrupt descriptor table (IDT) is
- This IDT contains pointers to places in memory which contain code to handle particular interrupts

Interrupt Descriptor Table (IDT)

- Table in memory pointed to by `idtr`
- Holds gate descriptors for interrupt service routines (ISRs) aka interrupt handlers
 - Gate descriptors are addresses to interrupt handlers

What happens when you execute an interrupt service routine (ISR)?



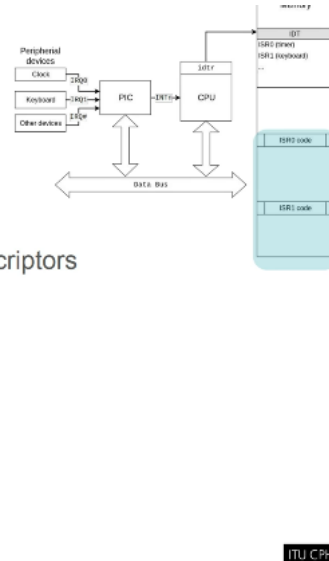
Basic ISR algorithm

handling an interrupt

interrupt service routines (ISRs)

Here is the basic ISR algorithm:

1. Save the state of interrupted procedure (pusha)
2. Save previous data segment
3. Reload data segment registers with kernel data descriptors
4. Acknowledge interrupt by sending EOI to PIC
5. [Do the work]
6. Restore data segment
7. Restore the state of interrupted procedure (popa)
8. Enable interrupts (sti)
9. Exit interrupt handler (with iret)

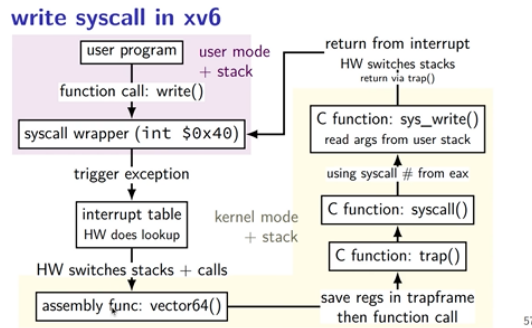


Interrupts in assembly

- Can be called with the `int` instruction or with syscalls

```
usys.S
(after macro replacement)
#include "syscall.h"
// ...
.globl write
write:
    /* 16 = SYS_write */
    movl $16, %eax
    /* 0x40 = T_SYSCALL */
    int $0x40
    ret
```

- You have to specify the id of the syscall in the rax / eax register
- You can set the interrupt descriptor table in C with the `lidt` function



Situations when an interrupt occurs

- A device needs attention (ex key press)
- The user program did something illegal (ex tries to access memory which it's not privileged to access)
- The user program asks the OS kernel for a service using a syscall
- In these cases, the flow of control is transferred from the user program to the OS kernel
- Interrupts can also be classified in
 - **Asynchronous events** - outside of the software's control, managed by hardware
 - Usually because a device triggers the interrupt
 - Examples: pressing ctrl C at the keyboard, arrival of a packet from a network, arrival of data from disk, hard reset interrupt (reset button), soft reset interrupt (ctrl alt del)
 - **Synchronous events (usually called exceptions)** - caused by software
 - **Traps** (intentional, returns control to the next instruction) : System calls, breakpoints
 - **Faults** (unintentional but possibly recoverable, either re-executes faulting instruction or aborts) : page faults (recoverable), floating point exceptions, protection faults (unrecoverable)

- **Aborts** (unintentional and unrecoverable, aborts current program) :
parity error, machine check, invalid memory reference

Booting

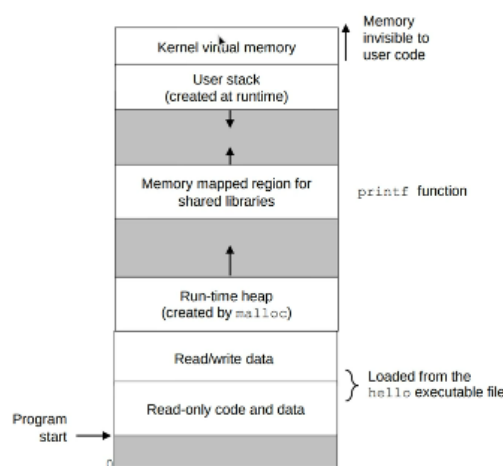
- Power button controls making a connection between two wires - when this connection is made the power supply unit (PSU) receives a signal
- The PSU is always listening for this connection
- When the PSU receives the signal it performs a self-test (makes sure that it is sending a good power signal to the computer), once its output has stabilized it will send a power good signal to the motherboard
- The motherboard is listening for this power good signal, when the signal is 1 then the motherboard will cause the computer to start
- When the computer starts the system firmware will run
 - The system firmware resides in read-only memory (ROM) on the motherboard
 - Examples of system firmware: BIOS, UEFI
 - There is a subset of memory addresses which are in ROM and not RAM
 - The first instruction the CPU will execute will be a `jmp` into firmware (called the "reset vector")
 - Then IDT and interrupt handlers are configured (the IDT will be replaced when the OS actually starts)
 - (Core boot is a more minimal version which loads the system firmware from ROM into CPU cache and executes it from there)
- Power-On Self-Test (POST)
 - Performed by the BIOS
 - It will talk to the different devices to make sure that they are working as expected
- Boot device has a bootloader - is split in partitions, at the very beginning it has a master boot record

- The device will look for this master boot record, if it finds that it will check for master boot code
- Operations the bootloader needs to do
 - Locate OS kernel on the disk
 - Load the kernel into computer memory
 - Start running the kernel code

Processes

- One of the OS' most important tasks is to manage processes
- Processes are primitive units for allocation of system resources ("unit of a running task on the system")
- Each process has its address space and one or more thread of control
- When you run a program a process representing the program's execution is created
- Processes are organized hierarchically - a process can create another process

Virtual Memory



Each process has its own address space.

All address spaces are structured in the same way.

- If a process tries to access the kernel virtual memory it is killed
- Transferring the thread of control from one process to another is called **context switching** and is managed by the OS kernel

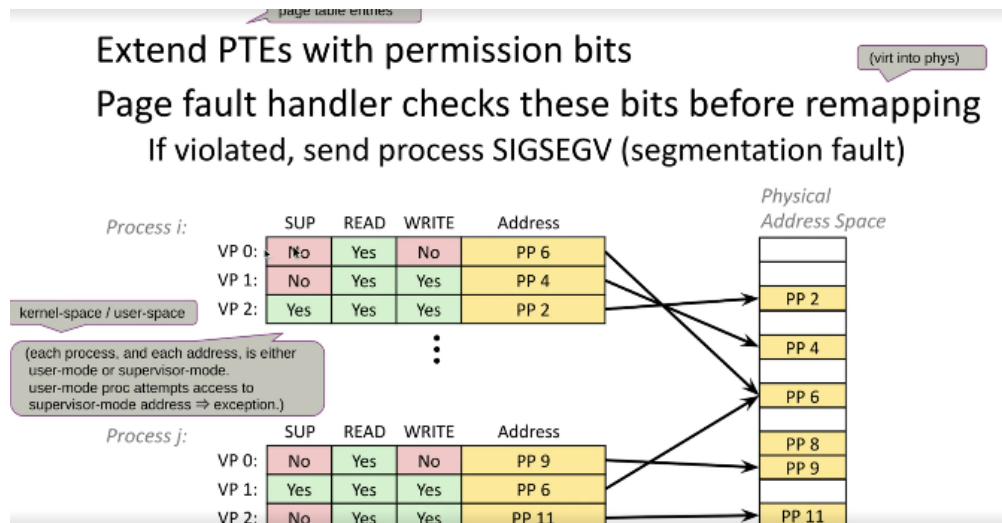
Context Switching

When a context switch is made the scheduler marks the task as interruptible, saves the process's task_struct and replaces the current tasks pointer with a pointer to the new process's task_struct, marked as running, restoring its memory access and register context.

- A process can be in two modes - user mode and supervisor mode
- Supervisor mode aka kernel mode is activated when kernel code needs to be ran, for example when doing I/O operations
- Supervisor mode is entered on syscall

How is a user-mode program prevented from messing with kernel stuff?

- Page table entries are extended with permission bits
- Permission bits indicate permissions such as read and write



- The SUP aka supervisor bit indicates whether an address is user mode or supervisor mode
- If a process in user mode attempts to access a resource in supervisor mode that will trigger a SIGSEGV (segmentation fault)
- At a lower level this is enforced using some special registers such as the CS register which contains bits which specify the current privilege level
 - These bits indicate whether the currently running process is in user mode or kernel mode
 - Normal code cannot modify this register
 - Only the int and syscall instructions can modify this register

Process state

- The state of each process running on the system is maintained by the kernel
- A `task_struct` is maintained
 - Contains the process descriptor which contains a `thread_struct`
 - Contains various attributes such as

Process context:

- 8KB / process in kernel space to store process descriptor `task_struct` (</linux/include/linux/sched.h>).

State:

```
#define TASK_RUNNING 0
#define TASK_INTERRUPTIBLE 1
#define TASK_UNINTERRUPTIBLE 2
#define TASK_ZOMBIE 4
#define TASK_STOPPED 8
```

Process ID

+ virtual memory info, file system info, open files, signal handlers, ...

- The thread of execution `thread_struct` (</linux/arch/x86/include/asm/processor.h>)
PC, registers, Fault info,

- Other info such as what the parent is, process id etc
- Process management in C (from libc)
 - Spawn process: `fork()`
 - Terminate process: `exit()`
 - Wait for process: `wait()`
 - Makes the current process stop until one of its children terminates (you can also specify a particular child process to wait for)
 - Execute a program within a process: `execve()`
- Zombie processes
 - When a process terminates it still consumes system resources
 - This happens if the process is still in RAM and the data structures about this process are still there
 - If a process dies it's the parent process' job to clean it away
 - If this doesn't happen then the zombie will be cleaned up by the `init` (nowadays `systemd`) process
 - Only need to explicitly clean zombies in long running processes such as shells and servers

Signals

- Notifies a process of an event, similar to exceptions and interrupts

Signals

A **signal** is a small message that notifies a process that an event of some type has occurred in the system

- akin to exceptions and interrupts
- sent from the kernel (sometimes at the request of another process) to a process
- signal type is identified by small integer ID's (1-30)
- only information in a signal is its ID and the fact that it arrived

ID	Name	Default Action	Corresponding Event
2	SIGINT	Terminate	Interrupt (e.g., ctrl-c from keyboard)
9	SIGKILL	Terminate	Kill program (cannot override or ignore)
11	SIGSEGV	Terminate & Dump	Segmentation violation
14	SIGALRM	Terminate	Timer signal
17	SIGCHLD	Ignore	Child stopped or terminated

Receiving a Signal

A destination process **receives** a signal. It is forced by the kernel to react in some way to the delivery of the signal

Three possible ways to react:

Ignore the signal (do nothing)

Terminate the process (with optional core dump)

Catch the signal by executing a user-level function called **signal handler**

Akin to a hardware exception handler being called in response to an asynchronous interrupt

Receiving Signals

Suppose kernel is returning from an exception handler and is ready to pass control to process p

Kernel computes $pnb = pending \ \& \ \sim blocked$

The set of pending nonblocked signals for process p

If ($pnb == 0$)

Pass control to next instruction in the logical flow for p

Else

Choose least nonzero bit k in pnb and force process p to **receive** signal k

The receipt of the signal triggers some **action** by p

Repeat for all nonzero k in pnb

Pass control to next instruction in logical flow for p

- Other than communicating with signals, processes can also communicate by having shared access to memory

Process scheduling

The refined set of MLFQ rules, spread throughout the chapter, are reproduced here for your viewing pleasure:

- **Rule 1:** If $Priority(A) > Priority(B)$, A runs (B doesn't).
- **Rule 2:** If $Priority(A) = Priority(B)$, A & B run in round-robin fashion using the time slice (quantum length) of the given queue.
- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue).
- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).
- **Rule 5:** After some time period S , move all the jobs in the system to the topmost queue.